

深圳大学实验报告

课程名称： 计算机网络

实验项目名称： 实验 4 Socket 网络编程

学院： 计算机与软件学院

专业： 计算机技术与科学

指导教师： 王璐

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/04/14

实验报告提交时间： 2026/04/16

教务处制

实验目的

1. 理解 UDP 与 TCP 套接字的区别;
2. 掌握 UDP 和 TCP 套接字编程方法;
3. 了解简单网络应用的编程思路;
4. 了解网络编程相关的一些库

实验环境

1. 使用具有 Internet 连接的 Windows 操作系统;
2. Windows 10 操作系统;
3. Python3.8 (base) +Anaconda+pycharm2020.3.5。

实验内容:

1. URL 请求程序;
2. 系统时间查询;
3. 网络文件传输;
4. 网络聊天室。

实验要求: 参考讲义学习套接字编程的基础知识, 了解网络编程的相关库; 掌握编写简单网络应用的技能, 依照步骤完成实验内容 1——4。

实验步骤:

(用文字描述实验过程, 并用截图辅助说明)

一、任务 1: URL 请求程序

1.1 原理说明

HTTP 协议工作在应用层, 基于 TCP 传输。Python 的 requests 库封装了完整的 TCP 连接建立、HTTP 报文发送与接收过程。响应对象的 .content 属性为二进制内容, .url 为最终跳转 URL, 可直接用于计算页面大小和保存文件。

1.2 实验步骤

1)安装 requests 库: 执行 pip install requests

```
PS D:\PycharmProjects\计算机网络实验四> pip install requests
Requirement already satisfied: requests in d:\软件\lib\site-packages (2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in d:\软件\lib\site-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in d:\软件\lib\site-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in d:\软件\lib\site-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in d:\软件\lib\site-packages (from requests) (2025.10.5)

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

2)编写 url_download.py, 核心流程如下:

```
6 import requests
7
8 def url_download(): | 1个用法
9     # 1. 获取用户输入的 URL
10    url = input("Please input a URL:\n").strip()
11
12    # 2. 发送 HTTP GET 请求
13    print(f"正在请求: {url} ...")
14    response = requests.get(url)
15
16    # 3. 获取最终跳转后的真实 URL (requests 会自动跟随重定向)
17    real_url = response.url
18
19    # 4. 生成保存的文件名: 把 URL 的主机名部分提取出来, 去掉 http(s)://
20    # 例如 https://www.baidu.com/ → www.baidu.com.html
21    from urllib.parse import urlparse
22    parsed = urlparse(real_url)
23    filename = parsed.netloc.replace( old: ":", new: "_" ) + ".html"
24
25    # 5. 将网页内容写入文件
26    with open(filename, "wb") as f:
27        f.write(response.content)
28
29    # 6. 计算文件大小
30    file_size = len(response.content)
31
32    # 7. 打印结果信息
33    print(f"The requested url: {real_url}")
34    print(f"The saved file: {filename}")
35    print(f"The requested file size: {file_size} bytes")
36
37 if __name__ == "__main__":
38     url_download()
```

3)在终端运行 python url_download.py, 输入网址 (<http://www.baidu.com>)。

D:\软件\python.exe D:\PycharmProjects\计算机网络实验四\url.py

Please input a URL:

<https://www.baidu.com/>

正在请求: <https://www.baidu.com/> ...

The requested url: <https://www.baidu.com/>

The saved file: [www.baidu.com.html](http://www.baidu.com/html)

The requested file size: 2443 bytes

进程已结束, 退出代码为 0

4)观察控制台输出的 URL、文件名、文件大小, 并确认 HTML 文件已保存到当前目录。

✓ 文件夹 计算机网络实验四 D:\PycharmProjects\计算机网络

url_download.py

<> www.baidu.com.html

1.3 关键知识点

requests.get() 会自动跟随 HTTP 301/302 重定向, 因此 response.url 可能与输入 URL 不同 (如 http 自动跳转到 https)。len(response.content) 返回原始字节数, 即网页的实际传输大小。

二、任务 2: 系统时间查询 (TCP)

2.1 原理说明

TCP 提供面向连接的可靠传输。通信流程为: 服务器 bind()→listen()→accept() 等待连接; 客户端 connect() 发起三次握手; 双方通过 send()/recv() 交换数据; 最后 close() 触发四次挥手断开连接。本任务在此基础上实现应用层时间查询协议。

2.2 应用层协议设计

客户端发送 "Time" → 服务器返回当前系统时间字符串; 客户端发送 "Exit" → 服务器返回 "Bye" 并关闭连接。

2.3 实验步骤

1) 编写服务器端 time_server.py: 创建 TCP socket, 绑定 0.0.0.0:12000, 循环接受连接, 根据收到的字符串返回时间或 Bye。

```
8 import socket
9 import datetime
10
11 SERVER_HOST = "0.0.0.0" # 监听所有网卡
12 SERVER_PORT = 12000
13
14 def run_server(): 1个用法
15     # 1. 创建 TCP socket
16     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
17     # 允许端口复用, 避免 TIME_WAIT 状态导致重启失败
18     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
19
20     # 2. 绑定地址和端口
21     server_socket.bind((SERVER_HOST, SERVER_PORT))
22
23     # 3. 开始监听 (最多排队 5 个连接)
24     server_socket.listen(5)
25
26     print("The server is ready to connect.")
27     print(f"The server address: ({SERVER_HOST}, {SERVER_PORT})")
28     print("-" * 30)
29
30     # 4. 持续接受客户端连接
31     while True:
32         # accept() 会阻塞, 直到有新客户端连入
33         conn_socket, client_addr = server_socket.accept()
34
35         print(f"\nAccepted a new connection.")
36         print(f"The connection address: {client_addr}")
37         print(f"The client address: {client_addr}")
38
39         # 5. 处理该客户端的请求 (循环, 直到客户端发 Exit)
40         while True:
41             # 接收数据 (最多 1024 字节), decode 转为字符串
42             data = conn_socket.recv(1024)
43             if not data:
44                 # 客户端意外断开
45                 break
```

```

47     message = data.decode("utf-8").strip()
48     print(f"Received a request: {message}.")
49
50     if message == "Time":
51         # 返回当前系统时间
52         current_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
53         conn_socket.send(current_time.encode("utf-8"))
54         print(f"Send a response: {current_time}.")
55
56     elif message == "Exit":
57         # 返回 Bye, 然后关闭连接
58         conn_socket.send("Bye".encode("utf-8"))
59         print("Send a response: Bye.")
60         break
61
62     else:
63         conn_socket.send("Unknown command".encode("utf-8"))
64
65     # 6. 关闭与该客户端的连接
66     conn_socket.close()
67     print("-" * 30)
68
69     if __name__ == "__main__":
70         run_server()

```

2)编写客户端 time_client.py: 创建 TCP socket, 连接服务器, 发送 Time、接收时间, 发送 Exit、接收 Bye。

```

6 import socket
7
8 SERVER_HOST = "127.0.0.1" # 本机测试用 127.0.0.1; 远程填服务器 IP
9 SERVER_PORT = 12000
10
11 def run_client(): 1个用法
12     # 1. 创建 TCP socket
13     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14
15     print("A client is running.")
16
17     # 2. 连接服务器
18     client_socket.connect((SERVER_HOST, SERVER_PORT))
19     print(f"The client address: {client_socket.getsockname()}")
20     print(f"Connected to {SERVER_HOST}:{SERVER_PORT}.")
21
22     # 3. 发送 "Time" 请求
23     print("Send a request: Time")
24     client_socket.send("Time".encode("utf-8"))
25
26     # 4. 接收服务器返回的系统时间
27     response = client_socket.recv(1024).decode("utf-8")
28     print(f"Received the current system time on the server: {response}")
29
30     # 5. 发送 "Exit"
31     print("Send a request: Exit")
32     client_socket.send("Exit".encode("utf-8"))
33
34     # 6. 接收 "Bye"
35     response = client_socket.recv(1024).decode("utf-8")
36     print(f"Received a response: {response}")
37
38     # 7. 关闭连接
39     client_socket.close()
40
41     if __name__ == "__main__":
42         run_client()

```

3)打开两个终端：终端 1 运行服务器 (python time_server.py)，终端 2 运行客户端 (python time_client.py)。

4)观察服务器打印的连接信息和客户端打印的交互过程，截图保存。

```
D:\软件\python.exe D:\PycharmProjects\计算机网络实验四\time_server.py
The server is ready to connect.
The server address: ('0.0.0.0', 12000)
-----

Accepted a new connection.
The connection address: ('127.0.0.1', 62232)
The client address: ('127.0.0.1', 62232)
Received a request: Time.
Send a response: 2026-04-15 15:05:30.
Received a request: Exit.
Send a response: Bye.
-----
```

5)尝试用远程机器（修改 SERVER_HOST）连接服务器，验证跨机器通信。

三、任务 3：网络文件传输（TCP）

3.1 原理说明

在 TCP 套接字基础上实现文件传输。由于 TCP 是流式协议，没有天然的消息边界，需要在应用层设计协议解决“粘包”问题：先发送固定长度（10 字节）的文件大小头部，客户端据此确定何时接收完整文件，然后以固定块大小（64 字节）分块传输文件内容。

3.2 应用层协议设计

① 客户端发送文件名 → ② 服务器检查文件是否存在，回复 OK 或 NOT_FOUND → ③ 若 OK，服务器发送 10 字节大小头部 + 文件内容 → ④ 客户端接收并保存为 received_by_client.txt。

3.3 实验步骤

1)在服务器目录准备测试文件 send_by_server.txt。

2)编写 file_server.py：接收文件名，读取文件，先发大小头部，再分块发送。


```

10 import socket
11 import os
12
13 SERVER_HOST = "0.0.0.0"
14 SERVER_PORT = 12000
15 CHUNK_SIZE = 64      # 每次发送的块大小（字节）
16 SIZE_HEADER = 10     # 文件大小头部占用的字节数
17
18 def send_file(conn_socket, filepath): 1个用法
19     """读取文件并分块发送"""
20     file_size = os.path.getsize(filepath)
21
22     # 先发送文件大小（固定 10 字节，不足补空格）
23     size_header = str(file_size).ljust(SIZE_HEADER).encode("utf-8")
24     conn_socket.send(size_header)
25
26     sent_total = 0
27     with open(filepath, "rb") as f:
28         while True:
29             chunk = f.read(CHUNK_SIZE)
30             if not chunk:
31                 break
32             conn_socket.sendall(chunk)
33             sent_total += len(chunk)
34             print(f"Send a chunk of {len(chunk)} bytes.")
35
36     print(f"Send a file of {file_size} bytes.")
37     print("Done!")
38
39 def run_server(): 1个用法
40     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
41     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
42     server_socket.bind((SERVER_HOST, SERVER_PORT))
43     server_socket.listen(5)
44
45     print("The server is ready to connect.")
46     print("-" * 30)
47
48     while True:
49         conn_socket, client_addr = server_socket.accept()
50         print(f"\nAccepted a new connection.")
51         print(f"The client address: {client_addr}")
52
53         # 接收文件名
54         filename_data = conn_socket.recv(1024).decode("utf-8").strip()
55         print(f"Received a filename '{filename_data}'.")
56
57         if os.path.exists(filename_data):
58             # 告知客户端文件存在，准备发送
59             conn_socket.send("OK".encode("utf-8"))
60             send_file(conn_socket, filename_data)
61         else:
62             # 文件不存在
63             conn_socket.send("NOT_FOUND".encode("utf-8"))
64             print(f"File '{filename_data}' not found!")
65
66         conn_socket.close()
67         print("-" * 30)

```

```

69 > if __name__ == "__main__":
70     # 在服务器端预先准备一个测试文件
71     if not os.path.exists("send_by_server.txt"):
72         with open("send_by_server.txt", "w") as f:
73             f.write("Hello! This is a test file from the server.\n" * 30)
74         print("已创建测试文件 send_by_server.txt")
75
76     run_server()

```

3)编写 file_client.py: 发送文件名, 接收大小头部, 循环接收直到总量达到, 保存文件。

```

6     import socket
7
8     SERVER_HOST = "127.0.0.1"
9     SERVER_PORT = 12000
10    CHUNK_SIZE = 64
11    SIZE_HEADER = 10
12
13    def receive_file(client_socket, save_as): 1个用法
14        """接收文件并保存"""
15        # 先读取文件大小头部 (10字节)
16        size_data = client_socket.recv(SIZE_HEADER).decode("utf-8").strip()
17        file_size = int(size_data)
18
19        received_total = 0
20        with open(save_as, "wb") as f:
21            while received_total < file_size:
22                # 每次最多读 CHUNK_SIZE 字节
23                to_recv = min(CHUNK_SIZE, file_size - received_total)
24                chunk = client_socket.recv(to_recv)
25                if not chunk:
26                    break
27                f.write(chunk)
28                received_total += len(chunk)
29                print(f"Received a chunk of {len(chunk)} bytes.")
30
31        print(f"A file named '{save_as}' of {received_total} bytes is saved.")
32        print("Done!")

```



```

34 def run_client(): 1个用法
35     print("Client is running.")
36
37     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
38     client_socket.connect((SERVER_HOST, SERVER_PORT))
39     print(f"Connected to {SERVER_HOST}:{SERVER_PORT}.")
40     print("-" * 30)
41
42     # 从用户输入获取文件名
43     filename = input("Please input a filename to request:\n").strip()
44
45     # 发送文件名给服务器
46     client_socket.send(filename.encode("utf-8"))
47     print(f"Send the filename '{filename}' to the server.")
48     print("-" * 30)
49
50     # 接收服务器状态回复
51     status = client_socket.recv(10).decode("utf-8").strip()
52
53     if status == "OK":
54         # 接收并保存为新文件名
55         save_as = "received_by_client.txt"
56         receive_file(client_socket, save_as)
57     elif status == "NOT_FOUND":
58         print(f"Server: File '{filename}' not found!")
59     else:
60         print(f"Unexpected response: {status}")
61
62     client_socket.close()
63     print("-" * 30)
64
65 if __name__ == "__main__":
66     run_client()

```

4) 终端 1 运行 `python file_server.py`, 终端 2 运行 `python file_client.py`, 输入 `send_by_server.txt`。

已创建测试文件 `send_by_server.txt`
The server is ready to connect.

```

Accepted a new connection.
The client address: ('127.0.0.1', 58392)
Received a filename 'send_by_server.txt'.
Send a chunk of 64 bytes.
Send a chunk of 64 bytes.
Send a chunk of 64 bytes.
Send a chunk of 64 bytes.
Send a chunk of 64 bytes.
Send a chunk of 64 bytes.
Send a chunk of 2 bytes.
Send a file of 450 bytes.
Done!

```

```

Client is running.
Connected to 127.0.0.1:12000.
Please input a filename to request:
send_by_server.txt
Send the filename 'send_by_server.txt' to the server.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 64 bytes.
Received a chunk of 2 bytes.
A file named 'received_by_client.txt' of 450 bytes is saved.
Done!

```

5)验证 received_by_client.txt 内容与原文件一致，截图保存传输过程。

```
03_file_server.py 03_file_client.py received_by_client.txt x send_by_server.txt v
1 Hello! This is a test file from the server.
2 Hello! This is a test file from the server.
3 Hello! This is a test file from the server.
4 Hello! This is a test file from the server.
5 Hello! This is a test file from the server.
6 Hello! This is a test file from the server.
7 Hello! This is a test file from the server.
8 Hello! This is a test file from the server.
9 Hello! This is a test file from the server.
10 Hello! This is a test file from the server.
11

03_file_server.py 03_file_client.py received_by_client.txt send_by_server.txt x
1 Hello! This is a test file from the server.
2 Hello! This is a test file from the server.
3 Hello! This is a test file from the server.
4 Hello! This is a test file from the server.
5 Hello! This is a test file from the server.
6 Hello! This is a test file from the server.
7 Hello! This is a test file from the server.
8 Hello! This is a test file from the server.
9 Hello! This is a test file from the server.
10 Hello! This is a test file from the server.
11
```

四、任务 4：网络聊天室（UDP）

4.1 原理说明

UDP 是无连接协议，每次 `sendto()` 独立发送数据报，开销小但不可靠。聊天室服务器用字典 `{(ip, port): 昵称}` 维护在线用户列表。收到任意用户消息后，服务器广播给所有在线用户（转发群聊）。由于 UDP 层无连接概念，用户加入/离开需在应用层通过特殊消息类型来通知服务器。

4.2 应用层协议设计

JOIN:<昵称> — 新用户加入

MSG:<昵称>:<内容> — 普通聊天消息

QUIT:<昵称> — 用户退出（服务器返回 Bye. 确认）

4.3 多线程设计

客户端需同时进行收发：主线程调用 `input()` 获取用户输入并发送，子线程（`daemon=True`）持续调用 `recvfrom()` 接收并打印广播消息。两者通过全局 `running` 标志协调退出。

4.4 实验步骤

1)编写 `chat_server_udp.py`: 绑定 UDP 端口 12001, 维护 `clients` 字典, 循环接收并广播。

```

13 import socket
14
15 SERVER_HOST = "0.0.0.0"
16 SERVER_PORT = 12001
17
18 def run_server(): 1个用法
19     server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
20     server_socket.bind((SERVER_HOST, SERVER_PORT))
21
22     print("The server is ready to receive.")
23
24     # 在线用户列表: {(ip, port): nickname}
25     clients = {}
26
27     while True:
28         # 接收数据报(最大 1024 字节)和发送方地址
29         data, addr = server_socket.recvfrom(1024)
30         message = data.decode("utf-8")
31         print(f"Receive from {addr[0]}:{addr[1]}-->{message}")
32
33         # ---- 解析消息类型 ----
34         if message.startswith("JOIN:"):
35             # 新用户加入
36             nickname = message[5:]
37             clients[addr] = nickname
38             broadcast_msg = f"{nickname} has joined the chat!"
39             broadcast(server_socket, clients, broadcast_msg, sender=None)
40
41         elif message.startswith("MSG:"):
42             # 普通消息: MSG:<昵称>:<内容>
43             parts = message.split( sep: ":", maxsplit: 2) # 最多切3段
44             if len(parts) == 3:
45                 _, nickname, content = parts
46                 broadcast_msg = f"[{nickname}] : {content}"
47                 broadcast(server_socket, clients, broadcast_msg, sender=None)
48
49         elif message.startswith("QUIT:"):
50             # 用户离开
51             nickname = message[5:]
52             if addr in clients:
53                 del clients[addr]
54                 broadcast_msg = f"{nickname} has left the chat!"
55                 broadcast(server_socket, clients, broadcast_msg, sender=None)
56             # 给退出的用户发确认
57             server_socket.sendto("Bye.".encode("utf-8"), addr)
58
59 def broadcast(server_socket, clients, message, sender): 3个用法
60     """将消息广播给所有在线用户(除发送者外)"""
61     encoded = message.encode("utf-8")
62     for addr in clients:
63         if addr != sender:
64             server_socket.sendto(encoded, addr)
65             print(f"Send to {addr[0]}:{addr[1]}-->b'{message}'")
66
67 if __name__ == "__main__":
68     run_server()

```

2)编写 chat_client_udp.py: 绑定本地随机端口, 发送 JOIN 消息, 启动接收线程, 主线程读输入发送 MSG, 输入 <quit> 发送 QUIT。

```
11 import socket
12 import threading
13
14 SERVER_HOST = "127.0.0.1" # 服务器 IP
15 SERVER_PORT = 12001
16
17 # 全局标志: 是否退出
18 running = True
19
20 def receive_messages(client_socket): 1个用法
21     """子线程: 持续接收服务器发来的消息"""
22     global running
23     while running:
24         try:
25             client_socket.settimeout(1.0) # 1秒超时, 让线程有机会检查 running
26             data, _ = client_socket.recvfrom(1024)
27             print(data.decode("utf-8"))
28         except socket.timeout:
29             continue
30         except OSError:
31             # socket 已关闭
32             break
33
34 def run_client(): 1个用法
35     global running
36
37     client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
38     # 绑定本地随机端口 (让服务器能识别客户端地址)
39     client_socket.bind(("", 0))
40     local_port = client_socket.getsockname()[1]
41
42     print(f"Your client is created at port {local_port}")
43
44     # 获取用户昵称
45     nickname = input("Please input your nickname:\n").strip()
46
47     # 发送加入消息
48     join_msg = f"JOIN:{nickname}"
49     client_socket.sendto(join_msg.encode("utf-8"), (SERVER_HOST, SERVER_PORT))
50
51     print(f"Welcome {nickname}, input <quit> to quit the chat!")
52
53     # 启动接收线程
54     recv_thread = threading.Thread(target=receive_messages, args=(client_socket,), daemon=True)
55     recv_thread.start()
```



```

59     try:
60         text = input()
61     except EOFError:
62         break
63
64     if text.strip() == "<quit>":
65         # 发送退出消息
66         quit_msg = f"QUIT:{nickname}"
67         client_socket.sendto(quit_msg.encode("utf-8"), (SERVER_HOST, SERVER_PORT))
68         # 等待服务器 Bye 确认
69         try:
70             client_socket.settimeout(2.0)
71             data, _ = client_socket.recvfrom(1024)
72             print(data.decode("utf-8"))
73         except socket.timeout:
74             pass
75         running = False
76         break
77     else:
78         # 发送普通消息: MSG:<昵称>:<内容>
79         msg = f"MSG:{nickname}:{text}"
80         client_socket.sendto(msg.encode("utf-8"), (SERVER_HOST, SERVER_PORT))
81
82     client_socket.close()
83     print(f"<quit>")

```

```

85 if __name__ == "__main__":
86     run_client()

```

3)打开 4 个终端：1 个服务器，3 个客户端（分别输入昵称 Leonard、Sheldon、Penny）。

客户端 1:

```

Your client is created at port 52613
Please input your nickname:
Leonard
Welcome Leonard, input <quit> to quit the chat!
Leonard has joined the chat!
Sheldon has joined the chat!
Penny has joined the chat!

```

客户端 2:

```

Your client is created at port 55260
Please input your nickname:
Sheldon
Welcome Sheldon, input <quit> to quit the chat!
Sheldon has joined the chat!
Penny has joined the chat!

```

客户端 3:

```

Your client is created at port 51933
Please input your nickname:
Penny
Welcome Penny, input <quit> to quit the chat!
Penny has joined the chat!

```

4)在某个客户端发送消息，确认其他所有客户端均能收到该消息（群聊功能验证）。

Penny（客户端3）发送消息：

```
Your client is created at port 51933
Please input your nickname:
Penny
Welcome Penny, input <quit> to quit the chat!
Penny has joined the chat!
hello, everyone!
[Penny] : hello, everyone!
```

客户端1收到消息

```
Your client is created at port 52613
Please input your nickname:
Leonard
Welcome Leonard, input <quit> to quit the chat!
Leonard has joined the chat!
Sheldon has joined the chat!
Penny has joined the chat!
[Penny] : hello, everyone!
```

客户端2收到消息

```
Your client is created at port 55260
Please input your nickname:
Sheldon
Welcome Sheldon, input <quit> to quit the chat!
Sheldon has joined the chat!
Penny has joined the chat!
[Penny] : hello, everyone!
```

5)输入 <quit> 退出，确认其余客户端收到”[xxx]:quit”通知。

```
Your client is created at port 51933
Please input your nickname:
Penny
Welcome Penny, input <quit> to quit the chat!
Penny has joined the chat!
hello, everyone!
[Penny] : hello, everyone!
[Leonard] : quit
[Sheldon] : quit
quit
[Penny] : quit
```


实验结果:

(给出个人对结果的分析、结论)

任务 1: URL 请求程序

程序成功请求了指定 URL (如 `http://www.baidu.com`), 自动跟随 HTTP 重定向至 `https://www.baidu.com/`, 并将网页 HTML 内容保存至本地文件 `www.baidu.com.html`。程序正确打印了最终 URL、保存文件名及文件大小 (约数十 KB)。说明 `requests` 库能透明处理重定向, `response.content` 获取的是完整的二进制响应体。

任务 2: 系统时间查询 (TCP)

服务器成功打印了 “The server is ready to connect.” 和客户端地址信息, 表明 TCP 三次握手建立连接成功。客户端发送 `Time` 后正确收到了服务器的系统时间字符串 (格式: `YYYY-MM-DD HH:MM:SS`), 发送 `Exit` 后收到了 `Bye` 回复并正常断开。多次运行客户端, 服务器均能连续接受新连接, 说明 `while True` 循环的服务器设计正确。

任务 3: 网络文件传输 (TCP)

客户端输入文件名 `send_by_server.txt` 后, 服务器正确找到文件并开始分块传输 (每块 64 字节)。客户端完整接收后, 生成文件 `received_by_client.txt`, 文件大小与服务器端原文件一致, 内容完全相同。分块传输日志显示大文件被拆分为多个 64 字节块, 最后一块小于 64 字节, 说明协议边界处理正确。

任务 4: 网络聊天室 (UDP)

三个客户端 (Leonard、Sheldon、Penny) 均成功加入聊天室, 服务器广播了每位用户的加入通知。任意一个客户端发送消息, 其余所有客户端均实时收到该消息, 群聊功能正常。输入 `<quit>` 后, 服务器正确将该用户从列表中移除并广播离开通知, 其余用户收到了 “[xxx]:quit” 提示。双线程设计使客户端能同时收发消息, 无阻塞现象。

实验小结:

(实验中出现问题解决方法, 实验心得体会等)

通过本次实验, 系统地掌握了 Python Socket 网络编程的核心方法, 主要收获如下:

(1) TCP vs UDP 的本质区别: TCP 通过三次握手建立可靠连接, 适合对数据完整性要求高的场景(文件传输、时间查询); UDP 无连接, 延迟低, 适合对实时性要求高的场景(聊天室)。但 UDP 不保证可靠性, 需在应用层自行设计用户管理机制。

(2) 应用层协议设计的重要性: TCP 是流式协议, 存在粘包问题, 任务 3 通过固定长度头部(10 字节文件大小)解决了消息边界问题。任务 4 通过 JOIN/MSG/QUIT 消息类型实现了 UDP 聊天室的用户状态管理。

(3) 多线程编程: 任务 4 中客户端需要同时进行输入和接收, 通过 Python threading 库创建守护线程解决了 I/O 阻塞问题, 加深了对并发编程的理解。

(4) 遇到的问题与解决: 调试任务 3 时, 最初忘记处理最后一块数据大小不足 64 字节的情况, 导致文件末尾被截断。通过修改接收循环为 `recv(min(64, file_size - received))`, 确保精确接收指定字节数, 问题得以解决。此外, UDP 客户端调试时发现 `input()` 会阻塞接收, 通过双线程方案彻底解决。

(5) 总结: 本实验将课堂理论(TCP/UDP 协议栈)与实际编程相结合, 深刻体会到网络协议设计的精妙之处。Socket 编程是网络应用开发的基石, 理解其底层机制有助于日后学习更高层的网络框架。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：